- When you call a method, Ruby goes *right* into the class of the receiver and then *up* the ancestors chain, until it either finds the method or reaches the end of the chain.
- When you include a module in a class, the module is inserted in the ancestors chain right *above* the class itself. When you prepend the module, it is inserted in the ancestors chain right *below* the class.
- When you call a method, the receiver takes the role of self.
- When you're defining a module (or a class), the module takes the role of self.
- Instance variables are always assumed to be instance variables of self.
- Any method called without an explicit receiver is assumed to be a method of self.
- Refinements are like pieces of code patched right over a class, and they override normal method lookup. On the other hand, a Refinement works in a limited area of the program: the lines of code between the call to using and the end of the file, or the end of the module definition.

Checked…checked…done! Now it's time to go home before your brain explodes with all the information you crammed into it today.